
Data Storage



Introduction

- ▶ Mobile apps usually need to store persistent data locally on user devices:
 - ▶ Temporary app state when closing
 - ▶ Cached data to avoid fetching the same data multiple times, or to allow the app to work offline
 - ▶ Sensitive user data that can't be transmitted over Internet
- ▶ Attn:
 - ▶ Mobile devices are not suitable to store large datasets (Android system: 6MB)
 - ➔ always try to limit the size of data

AsyncStorage Package

▶ Local storage

- ▶ Persistent
- ▶ Not secure (information need to be encrypted if necessary)

▶ Installation

- ▶ `npm install @react-native-async-storage/async-storage`

▶ Usage

- ▶ `import AsyncStorage from '@react-native-async-storage/async-storage';`

Storing Data

- ▶ `AsyncStorage.setItem(key, value)`
 - ▶ Asynchronous (returns a Promise)
 - ▶ Used both to add new data item and to modify existing item
- ▶ Examples
 - ▶

```
AsyncStorage.setItem("user-token", token)
  .then(() => {
    // ...
  })
  .catch(err => {
    // ...
  });
```
 - ▶

```
try {
  await AsyncStorage.setItem("user-token", token);
  // ...
} catch(err) {
  // ...
}
```

Reading Data

- ▶ `AsyncStorage.getItem(key)`
 - ▶ Asynchronous (returns a Promise)

- ▶ Examples

- ▶

```
AsyncStorage.getItem("user-token")
  .then(token => {
    // ...
  })
  .catch(err => {
    // ...
  });
```
- ▶

```
try {
  const token = await
    AsyncStorage.getItem("user-token");
  // ...
} catch(err) {
  // ...
}
```

Exercise

- ▶ Create an app with two tabbed screens that allow the user to enter a value and save it when clicking on a button on one screen, and to read and show this value back on the second screen.

Storing and Reading a Complex Object?

- ▶ Encode the object as a JSON string for storage:

- ▶ `const encoded = JSON.stringify(value);`
`await AsyncStorage.setItem(key, encoded);`

- ▶ Decode the JSON string when reading back:

- ▶ `const encoded = await AsyncStorage.getItem(key);`
`const value = JSON.parse(encoded);`

Exercise

- ▶ Modify the app in the previous exercise to allow the user to enter several values, but save/read them using a single storage key.

Get All Keys

- ▶ `AsyncStorage.getAllKeys()`

- ▶ Returns an array of all keys in the storage

- ▶ Example:

- ▶

```
try {  
    const keys = await AsyncStorage.getAllKeys();  
    // ...  
} catch(err) {  
    // ...  
}
```

Set Values for Multiple Keys at Once

- ▶ `AsyncStorage.multiSet(pairs)`
 - ▶ `pairs` is an array of `[key, value]` pairs

- ▶ **Example:**

```
▶ const pairs = [  
    ["key1", "value1"],  
    ["key2", "value2"],  
    ...  
];  
  
try {  
    await AsyncStorage.multiSet(pairs);  
    // ...  
} catch(err) {  
    // ...  
}
```

Get Values for Multiple Keys at Once

- ▶ `AsyncStorage.multiGet(keys)`
 - ▶ Returns corresponding key-value pairs found, stored as [key, value] array
- ▶ Example:
 - ▶

```
try {  
    const pairs = await  
        AsyncStorage.multiGet(keys);  
    // ...  
} catch(err) {  
    // ...  
}
```

Removing Items

- ▶ `AsyncStorage.removeItem(key)`
 - ▶ Removes 1 item
- ▶ `AsyncStorage.multiRemove(keys)`
 - ▶ Removes multiple items
- ▶ `AsyncStorage.clear()`
 - ▶ Removes whole storage
- ▶ **Examples:**
 - ▶ `await AsyncStorage.removeItem("my-key");`
 - ▶ `const keys = ["key1", "key2", ...];
await AsyncStorage.multiRemove(keys);`
 - ▶ `await AsyncStorage.clear();`